Payment Orchestrator

A standalone Go payment service. Single source of truth for who paid, for what, and whether access is still valid.

Stack

- **Go 1.26** — API server
- **PostgreSQL** — payments database
- **Razorpay** — subscription billing (INR only)
- **chi** — HTTP router
- **Plain HTML/CSS/JS** — UI (no build step)

First Time Setup

1. Install Go

Download from https://golang.org/dl — `go1.26.2.windows-amd64.msi`

Verify:

```
go version
# expected: go version go1.26.2 windows/amd64
```

2. Install PostgreSQL

Download from https://www.postgresql.org/download

Default port: `5432` Default user: `postgres`

3. Create the database and tables

```
psql -U postgres -f schema/payments.sql
```

This creates the `payments` database and all 4 tables automatically.

To verify:

```
psql -U postgres -d payments
\dt
```

You should see: `users`, `subscriptions`, `webhook_events`, `product_api_keys`

4. Configure environment

Copy `.env.example` to `.env` and fill in values:

```
cp .env.example .env
```

Minimum required to boot:

```
DB_PASSWORD=your_postgres_password
```

5. Install Go dependencies

```
go mod tidy
```

6. Run the server

```
go run main.go
```

Server starts on `http://localhost:8080`

Check health:

```
http://localhost:8080/health
```

Expected response:

```
{ "status": "ok", "mode": "skeleton" }
```

Skeleton mode is normal until Razorpay keys are added.

Environment Variables

Create a `.env` file in the project root. **Never commit this file.**

```
# Server
PORT=8080
SSO_DOMAIN=localhost:3000

# Database
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=your_password_here
DB_NAME=payments

# Razorpay - fill when keys arrive
RAZORPAY_KEY_ID=
RAZORPAY_KEY_SECRET=
RAZORPAY_WEBHOOK_SECRET=

# Razorpay Plan IDs - fill after creating plans in Razorpay dashboard
PLAN_ID_PRODUCT_1_MONTHLY=
```

```
PLAN_ID_PRODUCT_1_YEARLY=
PLAN_ID_PRODUCT_2_MONTHLY=
PLAN_ID_PRODUCT_2_YEARLY=
PLAN_ID_PRODUCT_3_MONTHLY=
PLAN_ID_PRODUCT_3_YEARLY=
PLAN_ID_PRODUCT_4_MONTHLY=
PLAN_ID_PRODUCT_4_YEARLY=
PLAN_ID_PRODUCT_5_MONTHLY=
PLAN_ID_PRODUCT_5_YEARLY=
PLAN_ID_MASTER_MONTHLY=
PLAN_ID_MASTER_YEARLY=

# Per-product API keys for /verify endpoint
API_KEY_PRODUCT_1=
API_KEY_PRODUCT_2=
API_KEY_PRODUCT_3=
API_KEY_PRODUCT_4=
API_KEY_PRODUCT_5=

# CORS - add SSO origin here when integrating
ALLOWED_ORIGINS=http://localhost:8080
```

Transferring to Another Machine

Do these steps in order:

1. Copy the entire project folder
2. **Do not copy `.env`** — create a fresh one on the new machine
3. Install Go 1.26 on the new machine
4. Install PostgreSQL on the new machine
5. Run `psql -U postgres -f schema/payments.sql` to recreate the DB
6. Fill `.env` with the new machine's DB password
7. Run `go mod tidy` to restore dependencies
8. Run `go run main.go`

`.env` is in `.gitignore` — it will never be pushed to GitHub. Each machine needs its own `.env`.

Changing Prices or Product Names

File: `core/finance/plans.go`

- **Price**: Change `AmountPaise` — 100 paise = 1. Example: 10 = `1000`

- **Product name**: Change `ProductName` only. Never change `ProductID` — it is stored in the database
- **Features**: Fill the `Features: []string{}` array per plan
- **Add a product**: Add 2 new entries (monthly + yearly). Add 2 matching env slots in `.env`

Restart the server after any change. UI reads from API — no UI changes needed.

API Endpoints

Method	Path	Purpose
GET	`/health`	Server status and mode
GET	`/plans`	All 12 SKUs
POST	`/subscribe`	Create Razorpay subscription
POST	`/webhook`	Razorpay payment confirmation (called by Razorpay directly)
GET	`/status/:id`	Poll subscription state
GET	`/verify`	Check user access (called by other products)

/verify usage

Called by other products to check if a user has access:

```
GET /verify?email=user@example.com&product=product_1
Header: X-API-Key: your_product_api_key
```

Response:

```
{
  "email": "user@example.com",
  "product": "product_1",
  "has_access": true,
  "role": "member",
  "reason": "active subscription"
}
```

Database Tables

Table	Purpose
`users`	Email, UUID, role (free/member/patron)
`subscriptions`	Who paid for what, Razorpay IDs, status, expiry
`webhook_events`	Idempotency log, prevents duplicate processing
`product_api_keys`	Per-product keys for /verify

Table	Purpose

Useful queries:

```
-- Check all users and roles
SELECT email, role, created_at FROM users;

-- Check all subscriptions
SELECT u.email, s.plan_id, s.status, s.current_end
FROM subscriptions s
JOIN users u ON s.user_id = u.id;

-- Check webhook log
SELECT event_id, event_type, processed_at FROM webhook_events;
```

Roles

Role	Access
`free`	No paid access
`member`	Access to one specific product
`patron`	Access to all products (Master Plan)

Role is assigned automatically by webhook when payment is confirmed. Role is revoked automatically when subscription is cancelled or expired.

When Razorpay Keys Arrive

1. Fill `.env` — `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`
2. Create 12 subscription plans in Razorpay dashboard
3. Fill the 12 `PLAN_ID_*` slots in `.env`
4. Implement the Razorpay API call in `api/subscriptions.go` where `TODO Phase 6` comment sits
5. Wire Razorpay modal in `ui/checkout/index.html`
6. Restart server — zero other code changes needed

Project Structure

```
payment-orchestrator/
```

```
main.go                          ← entry point, routes, DB connection
.env                             ← secrets (never commit)
.env.example                     ← safe template to commit
go.mod / go.sum                  ← dependencies
.gitignore

config/config.go                 ← loads all env vars
core/
    finance/
        plans.go                 ← all 12 SKUs - edit prices/names here
        signature.go             ← HMAC-SHA256 webhook verification
        coupons.go               ← stub, not active
    entitlement/
        roles.go                 ← access logic, role hierarchy

api/
    helpers.go                   ← writeJSON, writeError
    plans.go                     ← GET /plans
    subscriptions.go             ← POST /subscribe
    webhook.go                   ← POST /webhook
    status.go                    ← GET /status/:id
    verify.go                    ← GET /verify

store/
    db.go                        ← interface contract
    postgres/postgres.go         ← SQL implementation

schema/payments.sql              ← run this to create DB
middleware/cors.go               ← CORS, named origins only
ui/
    plans/index.html             ← plan selection UI
    checkout/index.html          ← checkout UI
```

What Is Parked (Not Active Yet)

- **Redis** — cache layer designed, not implemented
- **JWT** — cross-product auth designed, feature-flagged off
- **Coupons** — UI slot exists, backend stub only
- **Pro-rata upgrades** — not designed yet
- **SSO user_id** — email is current identity anchor, `user_id` column exists in DB ready for upgrade